Porting RTEMS on the Versal AI Edge Board

Alice Berta Francesco Danese

May 2026

Contents

1 Project data

- **Project supervisor:** Emilio Corigliano (emilio.corigliano@polimi.it)
- Group members:

Last and first name	Person code	Email address
Berta Alice	10889590	alice.berta@mail.polimi.it
Danese Francesco	10856896	francesco.danese@mail.polimi.it

- **Task subdivision:** Both members worked together on every phase of the project throughout its entire development: the Vivado hardware design, the RTEMS toolchain setup, the Vitis integration workflow, the debugging sessions, and the LED extension.
- **Source code and report:** https://github.com/fradane/RTEMS-x-Versal

2 Project description

What is the project about?

The project targets the AMD/Xilinx Versal AI Edge platform, hosted on a Trenz TE0950 system-on-module. The central goal was to port and run the RTEMS real-time operating system (RTOS) on the Real-Time Processing Unit (RPU) of the Versal device, specifically the two ARM Cortex-R5F cores operating in split mode, where each core runs independently rather than in the fault-tolerant lockstep configuration.

The project was conceived as an extension of previous work by another group who had successfully run a bare-metal application on the same board. Starting from that baseline, we set two progressive goals:

1. **Boot RTEMS on the RPU in split mode** and produce serial output, confirming that the RTOS is fully initialised and is running.
2. **Drive a user LED connected to the Programmable Logic (PL)** fabric from an RTEMS application, demonstrating that custom PL peripherals can be integrated and controlled by the RTOS through memory-mapped I/O.

Both goals were achieved. While a Board Support Package (BSP) for the Cortex-R5F on Versal already existed in the RTEMS codebase, almost no practical documentation was available covering the full toolchain setup from scratch. A secondary motivation for the project was therefore to document the process clearly enough to be reproducible by others. The complete and detailed guide is available at the project's GitHub repository [1].

Why is it important for the AOS course?

The project touches several core themes of the Advanced Operating Systems course:

- **RTOS architecture and execution model.** RTEMS is a real-time executive with a deterministic scheduler, a task-based execution model, and a minimal BSP abstraction layer between the OS and the hardware. Understanding how RTEMS initialises, how its tasks map to processor contexts, and how its drivers interface with peripherals is a direct exercise in OS internals.
- **Hardware-software co-design.** Making RTEMS work on this platform required designing the hardware in Vivado, configuring the SoC using Vitis, and building the OS from source against a specific BSP; these three entirely different toolchains must interoperate correctly at every step. This multi-layer integration is representative of the challenges faced in real embedded systems development.

- **Memory protection and low-level debugging.** A key obstacle in the project was an MPU (Memory Protection Unit) fault triggered by the RTEMS BSP when the application attempted to access a memory-mapped peripheral. Diagnosing and fixing this required understanding how the Cortex-R5F MPU works, how RTEMS configures it at startup, and how to patch the BSP to add the missing region descriptor.
- **Cross-compilation and toolchain management.** Building the RTEMS Source Builder, cross-compiling the kernel and an application for a bare-metal ARM target, and linking everything into a bootable image required working fluency with the GCC toolchain, build systems (waf), linker scripts, and binary formats.

2.1 Design and implementation

The solution is structured as a four-layer software stack:

Layer 1: Vivado hardware design. The hardware design was created in Vivado as a block design. The central IP is the CIPS (Control, Interfaces and Processing System), which represents the entire Versal Processing System. For the base configuration, the CIPS was connected to an AXI Network-on-Chip (NoC) to give the RPU access to DDR memory. For the LED extension, an additional AXI master port (`M_AXI_LPD`) was enabled on the CIPS and routed through an AXI SmartConnect to an AXI GPIO peripheral, whose single output pin was constrained to the physical LED pin on the TE0950 via an XDC constraints file. The design was synthesised, implemented, and exported as an XSA file (Xilinx Support Archive), which bundles the hardware description and the bitstream into a single handoff artifact.

Layer 2: Vitis platform and boot image assembly. Vitis Unified IDE consumed the XSA to generate a platform component, which compiled the mandatory Versal firmware: the PLM (Platform Loader and Manager) and the PSM (Platform Services Manager). A bootable PDI (Programmable Device Image) was then assembled by combining the PLM, PSM, and the RTEMS application ELF into a single binary. This image was programmed onto the board over JTAG without writing to flash.

Layer 3: RTEMS toolchain and BSP. RTEMS 7 was built from source using the RTEMS Source Builder, which compiles the full `arm-rtems7` cross-compilation toolchain automatically. The kernel was configured with the `arm/versal_rpu_split_0` BSP, which provides the startup code, interrupt controller initialisation, and timer configuration for RPU core 0 in split mode. Two non-default configuration options were needed: routing the console to UART1 instead of the default UART0 (see Section 3.4), and defining a non-cached memory window used during debugging. One modification to the BSP source itself was also required to add a missing MPU region descriptor covering the GPIO peripheral address space (see Section 3.4).

Layer 4: RTEMS application. Two RTEMS applications were written. The first is a minimal *Hello World*, that prints a message over UART and exits, whose purpose is solely to verify that RTEMS is initialised and the console driver is working. The second is a blinking-LED application: the RTEMS Init task writes to the AXI GPIO `DATA` register at the address assigned in the Vivado Address Editor (`0x80000000`) to toggle the LED on and off. The register is accessed as a volatile memory-mapped pointer, and a data synchronisation barrier (`dsb sy`) is issued after each write to guarantee the store reaches the peripheral. Both applications are available in the project repository [1].

3 Project outcomes

3.1 Concrete outcomes

- **RTEMS running on the Versal RPU in split mode.** RTEMS 7 boots successfully on Cortex-R5F core 0 of the Versal AI Edge device hosted on the Trenz TE0950 module. The RTOS scheduler is active and the RTEMS console driver produces output over UART1 at 115200 baud, visible on a host serial terminal.

- **PL LED controlled from RTEMS.** The user LED D1 on the TE0950 is toggled by an RTEMS task through an AXI GPIO peripheral mapped into the RPU's 32-bit address space at `0x80000000`. The LED blinks at a 2-second period for ten cycles before the application exits cleanly.
- **BSP patch: MPU region for GPIO peripheral.** A missing MPU region descriptor was identified in the RTEMS `arm/versal_rpu` BSP and added to `bsps/arm/versal_rpu/start/bspstart.c`. Without this patch, any RTEMS application that accesses a peripheral outside the pre-configured MPU regions triggers an immediate fault, like in the case of the blinking led.
- **Validated bare-metal dual-core communication framework.** As a stepping stone to verifying RTEMS execution, a bare-metal shared-memory communication scheme between both RPU cores was implemented and validated. Correct memory barrier usage was confirmed to be sufficient for inter-core coherency without disabling the data cache entirely.
- **Documentation.** A full technical report (separate from this document) describes both a step-by-step reproduction guide and a narrative of the development process with all relevant debugging findings. A potential further step is contributing this guide to the official RTEMS documentation, given the near-complete absence of practical material covering this toolchain setup from scratch for the Versal family.

3.2 Learning outcomes

- **Both members** deepened their understanding of the Xilinx/AMD Vivado toolchain: creating block designs from scratch, instantiating and configuring AXI IP cores, understanding address space assignment in heterogeneous SoCs, and writing pin constraint files. The project reinforced how hardware and software toolchains must be co-designed rather than treated as independent concerns.
- **Both members** learned how to approach a large open-source project (RTEMS) from scratch: navigating its build infrastructure (the RTEMS Source Builder and waf), reading and interpreting low-level C code such as BSP startup routines and MPU configuration tables, and performing register-level hardware debugging using XSDB without any application-level output to rely on. The hands-on debugging experience gave a much more concrete understanding of how memory protection works at the hardware level and how an OS configures it at boot time, a topic covered abstractly in lectures but now thoroughly understood in practice.
- **Both members** gained a deeper understanding of linker scripts: placing code and data sections at specific physical addresses, defining memory regions for the RPU address space, and diagnosing subtle link errors that arise when regions overlap or are sized incorrectly.

3.3 Existing knowledge

- The **Embedded Systems** and **Advanced Operating Systems** courses together provided the most directly applicable background. The relevant topics included: the differences between an RTOS and a general-purpose OS, the concept of memory-mapped I/O and peripheral drivers, the role of linker scripts in placing code and data at specific addresses.
- The **Computer Architecture** course helped when reasoning about processor memory models: understanding cache hierarchies, memory barriers, and the distinction between different memory types was necessary to correctly configure the MPU and to understand why memory barriers were needed when accessing memory-mapped peripherals.

3.4 Problems encountered

1: Conflicting linker scripts on dual-core bare-metal. When running two bare-metal applications simultaneously (one per RPU core), both applications printed the same output string. The cause was that Vitis generated identical default linker scripts for both cores, mapping both `.text` sections to the same base DDR address (`0x40000`). The

PLM therefore loaded the second ELF on top of the first, causing both cores to execute the same code. The fix was straightforward once the root cause was identified: the linker script for RPU 1 was updated to use a non-overlapping DDR base address (`0x10000000`). Neither member had encountered a multi-core linker conflict before, and the symptom (identical output from two supposedly independent programs) was initially quite confusing.

2: RTEMS hanging during UART initialisation. After assembling the first RTEMS boot image, no output appeared on the serial terminal. Since we had no way to distinguish a UART driver failure from a complete boot failure, we devised a side-channel verification: an RTEMS task wrote a sentinel value to a shared DDR address, while a bare-metal application on RPU 1 polled that address and printed a confirmation when it changed. This should have confirmed that RTEMS was actually running, but the RTOS seemed completely stuck: no visible writes to the shared address. Low-level XSDB debugging (reading the program counter and UART registers directly from the host debugger) revealed that RTEMS was spinning inside the UART0 initialisation routine, waiting for the transmit FIFO to drain. Since UART0 had never been enabled in the Vivado hardware design, the FIFO could never drain, and execution never progressed. The fix was a single line added to the RTEMS `config.ini` file, redirecting the BSP console to UART1 (`BSP_CONSOLE_MINOR = 1`).

3: AXI NoC placing GPIO addresses outside the 32-bit RPU space. When adding the AXI GPIO peripheral for the LED, connecting it through the existing AXI NoC caused Vivado's Address Editor to assign it an address above the 4 GB boundary. The ARM Cortex-R5F is a 32-bit core and cannot issue transactions to such addresses. The solution was to abandon the NoC path entirely and instead use a dedicated AXI LPD master port on the CIPS (`M_AXI_LPD`), routing it through an AXI SmartConnect directly to the GPIO peripheral. This kept the entire path within the LPD 32-bit address space and allowed the Address Editor to assign the GPIO a usable address at `0x80000000`.

4: RTEMS crashing on GPIO access due to missing MPU region. When the bare-metal LED application was replaced with the RTEMS version, the application crashed immediately upon attempting to write to the GPIO register. The root cause was that the RTEMS BSP for the Versal RPU configures the MPU at startup, and any access to an address not covered by a configured MPU region triggers a data abort fault on the Cortex-R5F. The address `0x80000000` was not covered by any of the default MPU regions in the BSP. The fix required modifying the BSP source file (`bsps/arm/versal_rpu/start/bspstart.c`) to add a new MPU region descriptor covering the LPD peripheral address range, configured as Device memory with read/write access and the Execute Never flag set. After recompiling RTEMS with this addition, the application ran correctly and the LED blinked as expected.

4 Honor Pledge

(**This part cannot be modified and it is mandatory to sign it**)

I/We pledge that this work was fully and wholly completed within the criteria established for academic integrity by Politecnico di Milano (Code of Ethics and Conduct) and represents my/our original production, unless otherwise cited.

I/We also understand that this project, if successfully graded, will fulfill part B requirement of the Advanced Operating System course and that it will be considered valid up until the AOS exam of Sept. 2026.

Group Students' signatures